

Issue #47 Readiness

08/09/2026 16:08:02

ok, what next can i do, remember what the goal is , #47

Thinking 🌐 Inspecting Repository State and Goal #47 ● Planning the...

ok, 58 is merged, lets do 62 (dont contradict anything already...

Thinking 🌐 Inspecting Repository Documentation and Issue Context ●...

Description Implement the backend database-access foundation,...

Thinking 🌐 Inspecting Repository Structure and Implementation Conte...

8a217afb-8a28-42fd-b5a9-5abd188debaa.png correct?? now give me...

Thinking 🌐 Inspecting Repository Structure and Step 9 ● Designing...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd ru...

Thinking 🌐 Inspecting Repository Environment Configuration ● Setting...

im confused with what im doing in the .env.test.. where am i putting wh...

Thinking 🌐 Inspecting PostgreSQL Configuration and Test Safety ●...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> psql --versio...

Thinking 🌐 Inspecting repository database setup documentation ●...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Service...

Thinking ● Choosing a database path I'm narrowing this to the simplest...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd ru...

Thinking ● Investigated database scripts, environment configuration, a...

yes pls

Thinking 🌐 Searching the web...

do i need this?? is there no other/better way? why do our tests require...

Thinking ● Planning integration tests I'm considering whether mocked...

do the docs not say anything about the tests anywhere? and how to ru...

Thinking 🌐 Inspecting Repository Documentation and Test Scripts ●...

ok, so whats my next step if those tests passed? also, our CI isnt workin...

Thinking ● Planning next steps Since the local tests are passing, I'm...

what about index in first commit?

Yes — include both relevant index.ts files in the first commit, but for...



ok, what next can i do, remember what the goal is , #47

8/9/2026, 3:09:20 PM



Thinking



Inspecting Repository State and Goal #47

- **Planning the next issue**

I'm treating the latest repository archive as the current source. The shortest path appears to be merging #58, then completing #62 before starting #47. I'm also checking whether #62 is already assigned; if so, #63 may be the next practical choice.

- **Planned the next issue**

- **Inspected repository state, history, dependencies, and referenced issues**

✓ Worked for 36s

Yes - keeping the actual goal in sight:

Your target is #47: public competition, fixture and participant read APIs.

Based on the **newest repo you gave me**, the best next thing for you is **#62**, not another random foundation issue.

Shortest route to #47

```
#58 Shared API contracts      ← your PR now
      ↓
#62 DB repository/transactions ← BEST NEXT ISSUE FOR YOU
      ↓
#47 Public read APIs          ← GOAL
```

There are also #46 and #63 around this path, but your current database already contains the core data #47 needs:

```
competition
fixture
team
person
fixture_team
fixture_squad
venue
```

So the biggest thing actually missing from the backend right now is **the layer that lets your handwritten Express API safely read that database.**

Your latest repo currently has:

```
apps/backend/src/
├─ auth/
├─ config/
├─ middleware/
├─ modules/
└─ routes/
```

but it **does not yet have a production repository/database-access layer.**

That is what #62 is for.

What I want you to do now

First, **finish #58 properly.**

If the PR is open:

1. Get the PR number.
2. Update your AI register's "PR pending" reference.
3. Push that little evidence update.
4. Get #58 reviewed/merged.

Don't begin #62 by branching off your #58 branch. Once #58 is merged:

```
git switch main
git pull origin main
```

Then check Gitea and make sure **#62 is not already assigned to / actively being worked on by somebody else.**

If it's free, assign yourself #62.

Then create:

```
git switch -c feat/62-database-repository-foundation
```

What #62 should accomplish

Think of #62 as building this missing section:

```
Express route
  ↓
controller
  ↓
service
  ↓
repository ← #62 gives us this foundation
  ↓
PostgreSQL
```

Then when we do #47:

```
GET /api/v1/competitions
GET /api/v1/fixtures
GET /api/v1/fixtures/:id
...
```

the endpoints can call repositories instead of putting SQL directly in Express routes.

Very important: still handwritten API

For #62 we should **not** do this:

```
supabase.from('fixture').select('*')
```

and definitely not expose Supabase's generated REST API.

The intended path remains:

```
Browser
  ↓ HTTP
OUR Express API
  ↓
OUR repository code
  ↓
PostgreSQL hosted by Supabase
```

The project brief requires the API itself to be designed and implemented by your group. [project_briefs.pdf](#)

#62 will probably contain things like

Something approximately like:

```
apps/backend/src/database/
├─ pool.ts
├─ transaction.ts
└─ index.ts

apps/backend/src/repositories/
└─ ...
```

or, depending on the architecture you already documented:

```
apps/backend/src/modules/<module>/
└─ *.repository.ts
```

We would likely establish:

- PostgreSQL connection/pool management;
- safe parameterised querying;
- transaction helper;
- rollback behaviour;
- clean connection release;
- repository conventions;
- tests for successful transactions;

- tests proving rollback on failure;
- configuration through environment variables;
- no credentials committed.

And importantly, **#62 should not implement #47's actual competition/fixture endpoints**. It only gives #47 the database foundation.

What about #46?

This needs coordination rather than you blindly taking it.

Your DB already has most of the entities that #47 needs, but the backlog still records:

#46 - competition, season, competitor, participant and fixture database schema.

Before #47, we need to know whether the team considers the current event-model schema to have **superseded/satisfied #46**, or whether somebody is still finishing #46.

So when you look at Gitea:

- if **#46 is closed/merged** → great;
- if **#46 is actively assigned** → let that person finish it;
- if **#46 is open and untouched** → tell me before taking it, because I want to compare its actual acceptance criteria against your current schema first. We may not need another pile of duplicate tables.

What about #63?

#63 is the representative seed/reference fixture data.

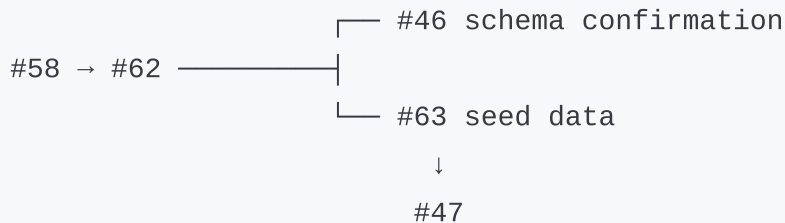
That is useful for #47 because once your endpoints exist, you'll want something like:

```
competition
├── fixture
│   ├── team A
│   ├── team B
│   └── players
```

to actually retrieve during tests.

But **I would choose #62 before #63** because #62 is the architectural blocker.

So:



#46/#63 can potentially be done by teammates while you're doing #62.

Therefore your next move

Yes: finish/merge #58, then take #62 if it is available.

And once #62 is merged, I expect we can finally start **#47 itself** rather than taking another unrelated issue.

If you send me the **actual Gitea description of #62** (screenshot or copy/paste), I'll take you through #62 exactly the same way we did #58 - step by step, with scope boundaries, files, tests, AI evidence, commit, and PR.

8/9/2026, 3:09:21 PM



ok, 58 is merged, lets do 62 (dont contradict anything already documented etc) is it a long one to do?

8/9/2026, 3:13:50 PM



Thinking



Inspecting Repository Documentation and Issue Context

- Inspected repository documentation and architecture for issue #62 scope

✓ Worked for 53s

Yes - **#62 is a medium-sized issue, not a huge one.**

It is a bit more technical than #58 because we are writing backend database infrastructure, but it should still be quite contained if we keep the scope tight. I'd expect roughly **4-7 small files plus tests**, not a massive feature.

Most importantly, I checked the current repo/docs you uploaded so we don't contradict existing decisions.

Your documented architecture already says:

- database access belongs **behind repositories**;
- controllers should stay thin;
- application data goes through the **handwritten Express backend**;
- PostgreSQL is accessed through the backend;
- multi-record acceptance must be **transactional: all records or none**;
- the accepted DB decision is to connect using the standard PostgreSQL `pg` driver via `DATABASE_URL`, not Supabase-generated data APIs.

So #62 should build exactly this:

```
Express route
  ↓
controller
  ↓
service
  ↓
repository
  ↓
pg / PostgreSQL
```

and **not** this:

```
supabase.from('fixture').select(...)
```

What #62 should probably contain

Something around:

```
apps/backend/src/database/
├─ pool.ts
├─ transaction.ts
└─ index.ts
```

plus tests such as:

```
apps/backend/tests/unit/database/
apps/backend/tests/database/
```

The core work is likely:

- reusable PostgreSQL connection pool;
- use existing `DATABASE_URL` ;
- keep TLS/certificate behaviour consistent with ADR-003;
- reusable transaction helper;
- automatic `BEGIN` ;
- automatic `COMMIT` on success;
- automatic `ROLLBACK` on failure;

- always release the checked-out client;
- repository/query typing conventions;
- parameterised SQL only;
- unit tests for transaction behaviour;
- database integration tests where appropriate.

You already have `pg` installed, and you already have a database transaction integration test proving PostgreSQL commit/rollback works. So we are **not starting from zero**.

What we should NOT do in #62

Do not implement:

- `/competitions`;
- `/fixtures`;
- `/participants`;
- actual #47 repositories;
- new database tables;
- migrations;
- seed data;
- Supabase-generated REST endpoints;
- statistics;
- submission logic.

Those belong to other issues.

So the path remains:

```
#58   
↓  
#62 ← NOW  
↓  
#47 ← YOUR ACTUAL GOAL
```

Before we write code

There is one thing I want from you because I only have the repository's planning entry for #62, not the actual Gitea issue acceptance criteria.

Paste the description of Gitea issue #62 here (or send a screenshot).

Then I'll compare it against the existing:

- ADR-003 database connection decision;
- backend module rules;
- testing foundation;
- system architecture;
- current `pg` setup;

and give you the **exact Step 1, Step 2, Step 3... implementation**, like we did for #58, without introducing anything that conflicts with the documented project.

8/9/2026, 3:13:51 PM



Description

Implement the backend database-access foundation, repository boundaries and transaction helpers used by application services.

Motivation

Submission, event and statistic operations must use consistent database access and atomic transactions rather than scattered queries.

Acceptance Criteria

- [] The backend has one documented database connection and configuration approach.
- [] Repository or data-access boundaries exist for the initial domain models.
- [] Transaction helpers support atomic multi-step operations.
- [] Errors are translated consistently without exposing credentials or raw sensitive details.
- [] Connection cleanup and failure handling are implemented.
- [] Automated integration tests cover successful commit and rollback.
- [] Backend documentation is updated and the work is merged through a Pull Request.

Dependencies

- Database migration tooling.
- Initial account and sport-domain schemas.

Evidence

- Integration test output.
- Database-access documentation.
- Linked Pull Request.

Project and Git Methodology

- Keep this issue in `Backlog` until its dependencies are understood, an assignee is selected and it meets the Definition of Ready.
- Create the branch from the latest `main` using `feat/<issue-number>-database-repositories`.
- Make small, meaningful commits using the approved Conventional Commit format.
- Reference this issue in commit bodies using `Refs # <issue-number>`.
- Run `npm run check` and all relevant tests before opening the Pull Request.
- Open a reviewed Pull Request whose description includes testing, known limitations and `Closes # <issue-number>`.
- Move the issue to `In Review` when the Pull Request is opened and to `Done` only after review, passing CI and merge.
- Where AI generated or substantially changed code, include the correct `Assisted-by` trailer and update `evidence/ai/ai-usage-register.csv`.

AI Declaration

> The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

8/9/2026, 3:17:15 PM



Thinking



Inspecting Repository Structure and Implementation Context

- **Reviewing database configuration**

I'm checking how CommonJS compilation, development execution, and path resolution affect certificate loading. I'm also considering whether the database URL belongs in centralized environment validation without breaking existing tests or startup behavior.

- **Reviewed database configuration**



Inspecting Backend Bootstrap and Test Configuration

- **Planning implementation steps**

I'm treating this as a medium-sized change, likely best handled incrementally. I'll start from the latest main branch, verify prerequisite schemas, then align the pool, environment settings, TLS certificate handling, migrations, repositories, and tests.

- **Planned implementation steps**



Inspecting MkDocs database documentation navigation

- **Inspected repository architecture, history, database configuration, tests, and documentation structure**

✓ Worked for 2m 52s

Yes - **#62 is a medium issue**, but it is very manageable if we keep it tightly scoped.

It is more coding than #58, but it should not become a monster. I would treat it as **two meaningful commits**:

Commit 1

Database pool + transaction + error foundation

Commit 2

Initial repository boundaries + integration tests + docs

And importantly, this gets you directly closer to **#47**.

First: the dependencies are understood

From the repository you gave me, the dependencies #62 asks for are already present:

- migration tooling exists;
- `app_user` exists;
- `competition`, `fixture`, `team`, `person`, etc. exist;
- there is already a dedicated test database setup;
- there are already raw PostgreSQL commit/rollback integration tests.



So #62 is ready to move out of `Backlog`.

The existing architecture also already tells us exactly how to implement it:

```
Frontend
  ↓
Handwritten Express API
  ↓
Service
  ↓
Repository
  ↓
pg
  ↓
PostgreSQL
```

We will **not use Supabase's Data API** or `supabase.from(...)` for application data.

For database access we will follow ADR-003:

```
pg driver
DATABASE_URL
Supavisor session-mode pooler
TLS certificate verification
```

The later Supabase Auth decision does not change this database boundary.

Step 1 - update Gitea first

Assign **#62 to yourself**.

Move it from:

```
Backlog
```

to whichever column your methodology uses for active work, probably:

```
In Progress
```

Since its dependencies are now understood and you have selected an assignee, it meets the condition in the issue description.

Step 2 - make sure you start from the merged #58 main

Run:

```
git switch main
git pull origin main
git status
```

You want:

```
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Then create the **exact branch name required by #62**:

```
git switch -c feat/62-database-repositories
```

Verify:

```
git branch --show-current
```

Expected:

```
feat/62-database-repositories
```

Step 3 - what we are going to add

Do **not** start modifying random existing scripts.

We are going to add an application database layer under:

```
apps/backend/src/database/
```

I recommend this structure:

```
apps/backend/src/database/
├─ errors.ts
```

```
|— pool.ts
|— query.ts
|— transaction.ts
└— index.ts
```

Then repository boundaries in the existing module structure:

```
apps/backend/src/modules/
|— competitions/
|   └— competition.repository.ts
|— fixtures/
|   └— fixture.repository.ts
└— participants/
    └— participant.repository.ts
```

Why these three?

Because they are genuine initial domain models **and they are exactly what #47 will need next**.

We are **not** creating HTTP endpoints in #62.

Step 4 - create the database error abstraction

Create:

```
apps/backend/src/database/errors.ts
```

Put:

```
export type DatabaseErrorCode =
  | 'DATABASE_UNAVAILABLE'
  | 'DATABASE_CONFLICT'
  | 'DATABASE_REFERENCE_ERROR'
  | 'DATABASE_CONSTRAINT_ERROR'
  | 'DATABASE_OPERATION_FAILED'
  | 'DATABASE_TRANSACTION_FAILED';

export class DatabaseAccessError extends Error {
  constructor(
    public readonly code: DatabaseErrorCode,
```

```

    message: string,
    options?: ErrorOptions,
  ) {
    super(message, options);
    this.name = 'DatabaseAccessError';
  }
}

function postgresErrorCode(error: unknown): string | undefined {
  if (
    typeof error === 'object' &&
    error !== null &&
    'code' in error &&
    typeof error.code === 'string'
  ) {
    return error.code;
  }

  return undefined;
}

export function translateDatabaseError(
  error: unknown,
): DatabaseAccessError {
  if (error instanceof DatabaseAccessError) {
    return error;
  }

  const postgresCode = postgresErrorCode(error);

  if (postgresCode?.startsWith('08')) {
    return new DatabaseAccessError(
      'DATABASE_UNAVAILABLE',
      'The database is currently unavailable.',
      { cause: error },
    );
  }

  switch (postgresCode) {
    case '23505':
      return new DatabaseAccessError(

```

```

        'DATABASE_CONFLICT',
        'The database operation conflicts with existing data.',
        { cause: error },
    );

case '23503':
    return new DatabaseAccessError(
        'DATABASE_REFERENCE_ERROR',
        'The database operation references unavailable data.',
        { cause: error },
    );

case '23502':
case '23514':
    return new DatabaseAccessError(
        'DATABASE_CONSTRAINT_ERROR',
        'The database rejected invalid data.',
        { cause: error },
    );

default:
    return new DatabaseAccessError(
        'DATABASE_OPERATION_FAILED',
        'The database operation could not be completed.',
        { cause: error },
    );
}
}

```

Why this matters

A raw PostgreSQL failure can contain things like:

```

SQL fragments
constraint names
table information
connection information
driver details

```

Those should not escape to API consumers.

Instead, services receive a controlled error such as:

```
DATABASE_CONFLICT
```

with a safe message.

That satisfies:

Errors are translated consistently without exposing credentials or raw sensitive details.

Step 5 - create the query boundary

Create:

```
apps/backend/src/database/query.ts
```

Use:

```
import type {
  QueryResult,
  QueryResultRow,
} from 'pg';

import { translateDatabaseError } from '../errors';

export interface QueryExecutor {
  query<Row extends QueryResultRow = QueryResultRow>(
    text: string,
    values?: unknown[],
  ): Promise<QueryResult<Row>>;
}

export async function executeQuery<
  Row extends QueryResultRow,
>(
  executor: QueryExecutor,
  text: string,
  values: unknown[] = [],
): Promise<QueryResult<Row>> {
```

```

try {
  return await executor.query<Row>(text, values);
} catch (error) {
  throw translateDatabaseError(error);
}
}

```

This is important because later repositories do not each invent their own error handling.

They all use:

```
executeQuery(...)
```

Step 6 - create the PostgreSQL pool

Create:

```
apps/backend/src/database/pool.ts
```

This must follow ADR-003 rather than inventing another connection method.

Use:

```

import { readFileSync } from 'node:fs';
import { resolve } from 'node:path';

import {
  Pool,
  type PoolConfig,
} from 'pg';

import {
  DatabaseAccessError,
  translateDatabaseError,
} from './errors';

let applicationPool: Pool | undefined;

export interface DatabasePoolOptions {
  connectionString: string;
}

```

```

    ssl?: PoolConfig['ssl'];
    max?: number;
  }

export function createDatabasePool(
  options: DatabasePoolOptions,
): Pool {
  const pool = new Pool({
    connectionString: options.connectionString,
    ssl: options.ssl,
    max: options.max ?? 10,
    connectionTimeoutMillis: 10_000,
    idleTimeoutMillis: 30_000,
  });

  pool.on('error', () => {
    // Do not expose the raw driver error because it may contain
    // database or connection details.
    console.error('Unexpected PostgreSQL pool error.');
```

```
  });
```

```
  return pool;
}
```

```
function loadApplicationDatabaseUrl(): string {
  const connectionString = process.env.DATABASE_URL;
```

```
  if (!connectionString?.trim()) {
    throw new DatabaseAccessError(
      'DATABASE_UNAVAILABLE',
      'DATABASE_URL is not configured.',
    );
  }
```

```
  return connectionString;
}
```

```
function loadApplicationTlsConfiguration(): PoolConfig['ssl'] {
  const certificatePath = resolve(
    __dirname,
    '../../certs/supabase-ca.crt',
```

```

);

return {
  rejectUnauthorized: true,
  ca: readFileSync(certificatePath, 'utf8'),
};
}

export function getDatabasePool(): Pool {
  if (!applicationPool) {
    applicationPool = createDatabasePool({
      connectionString: loadApplicationDatabaseUrl(),
      ssl: loadApplicationTlsConfiguration(),
    });
  }

  return applicationPool;
}

export async function closeDatabasePool(): Promise<void> {
  if (!applicationPool) {
    return;
  }

  const pool = applicationPool;
  applicationPool = undefined;

  try {
    await pool.end();
  } catch (error) {
    throw translateDatabaseError(error);
  }
}

```

Important

This is **not a Supabase API**.

It is:

pg → PostgreSQL connection string

exactly as ADR-003 requires.

Step 7 - add the transaction helper

Create:

apps/backend/src/database/transaction.ts

Use:

```
import type {
  Pool,
  PoolClient,
} from 'pg';

import {
  DatabaseAccessError,
  translateDatabaseError,
} from '../errors';

export async function withTransaction<T>(
  pool: Pool,
  operation: (client: PoolClient) => Promise<T>,
): Promise<T> {
  let client: PoolClient;

  try {
    client = await pool.connect();
  } catch (error) {
    throw translateDatabaseError(error);
  }

  let transactionStarted = false;

  try {
    await client.query('BEGIN');
    transactionStarted = true;
```

```

    const result = await operation(client);

    await client.query('COMMIT');

    return result;
} catch (error) {
    if (transactionStarted) {
        try {
            await client.query('ROLLBACK');
        } catch (rollbackError) {
            throw new DatabaseAccessError(
                'DATABASE_TRANSACTION_FAILED',
                'The database transaction failed and rollback could not be
confirmed.',
                {
                    cause: new AggregateError([
                        error,
                        rollbackError,
                    ]),
                },
            );
        }
    }
}

// Preserve application/domain errors thrown by the operation.
// PostgreSQL errors should normally already have been translated
// by executeQuery().
throw error;
} finally {
    client.release();
}
}

```

This directly gives us:

```

BEGIN
  ↓
multiple repository operations
  ↓
COMMIT

```

or:

```
BEGIN
  ↓
something fails
  ↓
ROLLBACK
  ↓
release connection
```

Exactly what #62 asks for.

Step 8 - export the database foundation

Create:

```
apps/backend/src/database/index.ts
```

Put:

```
export * from './errors';
export * from './pool';
export * from './query';
export * from './transaction';
```

Step 9 - repository boundaries

This is the part that gets us closer to **#47**.

We should create repository boundaries for:

```
competition
fixture
participant
```

but **no routes/controllers yet**.

The repository layer will be the only place that knows SQL such as:

```
SELECT ...
```

```
FROM fixture
```

Later #47 becomes:

```
GET /fixtures
  ↓
fixture service
  ↓
fixture repository
```

rather than embedding SQL in Express.

I would make these next:

```
apps/backend/src/modules/competitions/competition.repository.ts
apps/backend/src/modules/fixtures/fixture.repository.ts
apps/backend/src/modules/participants/participant.repository.ts
```

I'll give you the exact contents for these after we confirm Steps 1-8 compile, because they should line up precisely with the **post-#58 contracts and current schema**.

Step 10 - improve shutdown cleanup

We will also update:

```
apps/backend/src/index.ts
```

so when the API receives:

```
SIGINT
SIGTERM
```

it closes:

```
HTTP server
  ↓
PostgreSQL pool
```

rather than leaving database connections hanging.

That satisfies another #62 acceptance criterion:

Connection cleanup and failure handling are implemented.

Step 11 - replace the old transaction tests

You already have:

```
apps/backend/tests/database/transaction.database.test.ts
```

That is useful, but currently it tests raw:

```
client.query('BEGIN');  
client.query('ROLLBACK');
```

For #62 we need it to test **our transaction helper**.

So we will update that test to prove:

```
withTransaction()  
  ↓  
successful callback  
  ↓  
COMMIT ✅
```

and:

```
withTransaction()  
  ↓  
callback throws  
  ↓  
ROLLBACK ✅
```

That directly satisfies:

Automated integration tests cover successful commit and rollback.

Step 12 - documentation

We'll add something like:

```
docs/database/access.md
```

It will document:

```
DATABASE_URL
↓
Supervisor session pooler
↓
pg Pool
↓
Repository
↓
Service
```

and explicitly state:

```
No Supabase Data API
No PostgREST
No direct frontend database access
No SQL in controllers
Parameterised queries only
Transactions via withTransaction()
```

Then add it to `mkdocs.yml`.

Step 13 - likely commits

Because your methodology explicitly asks for **small, meaningful commits**, don't make one giant commit.

I suggest:

Commit 1

```
feat(database): add database pool and transaction helpers
```

```
Refs #62
```

```
Assisted-by: ChatGPT-Web[GPT-5.6 Sol]
```

Containing roughly:

```
database/errors.ts
database/query.ts
database/pool.ts
database/transaction.ts
database/index.ts
index.ts cleanup
```

Commit 2

```
feat(database): add initial repository boundaries
```

Refs #62

Assisted-by: ChatGPT-Web[GPT-5.6 Sol]

Containing:

```
competition.repository.ts
fixture.repository.ts
participant.repository.ts
transaction integration tests
database access docs
mkdocs.yml
AI evidence updates
```

That follows the issue's methodology much better than one enormous commit.

How long is it?

Relative to #58:

#58	■■■■■
#62	■■■■■■■
#47	■■■■■■■■■+

So #62 is a little bigger, but **not dramatically bigger**.

A lot of the difficult decisions have already been made for us:

DB provider	✓ decided
driver	✓ pg
connection variable	✓ DATABASE_URL
pooler mode	✓ session
TLS policy	✓ verify certificate
repository architecture	✓ documented
test database	✓ already exists
commit/rollback test	✓ baseline exists
schema	✓ exists

We're mostly turning those existing decisions into reusable application code.

Do this first

Run only:

```
git switch main
git pull origin main
git status
git switch -c feat/62-database-repositories
git branch --show-current
```

Then create:

```
apps/backend/src/database/errors.ts
apps/backend/src/database/query.ts
apps/backend/src/database/pool.ts
apps/backend/src/database/transaction.ts
apps/backend/src/database/index.ts
```

using the code above.

Don't create the repositories yet. After those five files are in, run:

```
npm.cmd run typecheck --workspace=@sport-analytics/backend
npm.cmd run lint --workspace=@sport-analytics/backend
```

If you send me the output, we'll do the repository part next and keep #62 controlled rather than debugging ten changes at once.

8/9/2026, 3:17:16 PM



8a217afb-8a28-42fd-b5a9-5ab...

correct?? now give me from astep 9 with actual steps of what to do

8/9/2026, 3:26:19 PM



Thinking



Inspecting Repository Structure and Step 9

- **Designing repositories**

I'm shaping the repository layer around the existing schema and query wrapper. I'll keep it data-access-only, preserve PostgreSQL identifiers as strings, support transaction-compatible executors, and cover competitions, fixtures, and fixture participants.

- **Designed repositories**

- **Refining data types**

I'm keeping nullable fixture metadata explicit, casting date fields to text for predictable API values, and joining venues only where useful. Participant access will use existing squad and person tables rather than inventing new schema.

- **Refined data types**



Inspecting Transaction Error Translation and Configuration

- **Inspected repository, database, backend configuration, and deployment setup**

✓ Worked for 5m 28s

Yes — **the structure in your screenshot is correct.** You have the right separation:

```
src/  
├─ database/  
└─ errors.ts
```

```

|   |─ index.ts
|   |─ pool.ts
|   |─ query.ts
|   └─ transaction.ts
|
└─ modules/
    |─ competitions/
    |   └─ competition.repository.ts
    |─ fixtures/
    |   └─ fixture.repository.ts
    └─ participants/
        └─ participant.repository.ts

```

That follows the existing documented rule that **database access belongs behind repositories**, and it keeps us on the path to **#47** without actually implementing #47 yet.

One small correction before Step 9: replace your current `transaction.ts` with the version below. The earlier version I gave you could allow a raw `BEGIN / COMMIT` database error through. This version routes those through the same safe error translator.

1. **Correct** `apps/backend/src/database/transaction.ts` .

```

import type { Pool, PoolClient } from 'pg';

import { DatabaseAccessError, translateDatabaseError } from '../errors';
import { executeQuery } from '../query';

export async function withTransaction<T>(
  pool: Pool,
  operation: (client: PoolClient) => Promise<T>,
): Promise<T> {
  let client: PoolClient;

  try {
    client = await pool.connect();
  } catch (error) {
    throw translateDatabaseError(error);
  }
}

```

```

let transactionStarted = false;

try {
  await executeQuery(client, 'BEGIN');
  transactionStarted = true;

  const result = await operation(client);

  await executeQuery(client, 'COMMIT');
  transactionStarted = false;

  return result;
} catch (error) {
  if (transactionStarted) {
    try {
      await executeQuery(client, 'ROLLBACK');
    } catch (rollbackError) {
      throw new DatabaseAccessError(
        'DATABASE_TRANSACTION_FAILED',
        'The database transaction failed and rollback could not be
confirmed.',
        {
          cause: new AggregateError([error, rollbackError]),
        },
      );
    }
  }

  throw error;
} finally {
  client.release();
}
}

```

This means:

```

pool.connect()
  ↓
BEGIN
  ↓

```

```
service/repository work
↓
COMMIT
```

and if anything fails:

```
failure
↓
ROLLBACK
↓
release client
```

No Supabase-generated API is involved.

2. **Step 9: fill** `competition.repository.ts` .

Open:

```
apps/backend/src/modules/competitions/competition.repository.ts
```

Paste:

```
import { executeQuery, getDatabasePool, type QueryExecutor } from
'../../database';

export interface CompetitionRecord {
  competitionId: string;
  name: string;
}

type CompetitionRow = CompetitionRecord;

export async function listCompetitions(
  executor: QueryExecutor = getDatabasePool(),
): Promise<CompetitionRecord[]> {
  const result = await executeQuery<CompetitionRow>(
    executor,
    `
    SELECT
      competition_id::text AS "competitionId",
      name
```

```

        FROM competition
        ORDER BY name ASC, competition_id ASC
    ` ,
);

return result.rows;
}

export async function findCompetitionById(
    competitionId: string,
    executor: QueryExecutor = getDatabasePool(),
): Promise<CompetitionRecord | null> {
    const result = await executeQuery<CompetitionRow>(
        executor,
        `
            SELECT
                competition_id::text AS "competitionId",
                name
            FROM competition
            WHERE competition_id = $1
        ` ,
        [competitionId],
    );

    return result.rows[0] ?? null;
}

```

Notice:

```
competition_id::text
```

That deliberately maintains the #58 decision that PostgreSQL `bigint` identifiers cross the JSON/application boundary as **strings**.

And:

```
WHERE competition_id = $1
```

is parameterised SQL. We are **not** inserting user input into SQL strings.

3. **Fill** `fixture.repository.ts` .

Open:

```
apps/backend/src/modules/fixtures/fixture.repository.ts
```

Paste:

```
import { executeQuery, getDatabasePool, type QueryExecutor } from
'../../database';

export interface FixtureRecord {
  fixtureId: string;
  sourceRef: string;
  competitionId: string | null;
  season: string;
  matchType: string;
  teamType: string;
  gender: string;
  ballsPerOver: number;
  scheduledOvers: number | null;
  venueId: string | null;
  startDate: string;
  endDate: string;
}

type FixtureRow = FixtureRecord;

export async function findFixtureById(
  fixtureId: string,
  executor: QueryExecutor = getDatabasePool(),
): Promise<FixtureRecord | null> {
  const result = await executeQuery<FixtureRow>(
    executor,
    `
    SELECT
      fixture_id::text AS "fixtureId",
      source_ref AS "sourceRef",
      competition_id::text AS "competitionId",
      season,
      match_type AS "matchType",
      team_type AS "teamType",
      gender,
  `
```

```

        balls_per_over AS "ballsPerOver",
        scheduled_overs AS "scheduledOvers",
        venue_id::text AS "venueId",
        start_date::text AS "startDate",
        end_date::text AS "endDate"
    FROM fixture
    WHERE fixture_id = $1
`,
    [fixtureId],
);

return result.rows[0] ?? null;
}

```

We're deliberately **not** implementing `listFixtures`, filters, cursors, HTTP responses, etc. yet.

Those belong to **#47**.

This repository only establishes the database boundary.

4. **Fill** `participant.repository.ts`.

Your database does **not** have a `participant` table. Do not create one.

Your existing model represents players using:

```

person
+
fixture_squad
+
team

```

So the repository translates that database representation into the application's idea of a fixture participant.

Open:

```
apps/backend/src/modules/participants/participant.repository.ts
```

Paste:

```

import { executeQuery, getDatabasePool, type QueryExecutor } from
'../../database';

export interface FixtureParticipantRecord {
  participantId: string;
  displayName: string;
  teamId: string;
  teamName: string;
  role: string | null;
}

type FixtureParticipantRow = FixtureParticipantRecord;

export async function listFixtureParticipants(
  fixtureId: string,
  executor: QueryExecutor = getDatabasePool(),
): Promise<FixtureParticipantRecord[]> {
  const result = await executeQuery<FixtureParticipantRow>(
    executor,
    `
      SELECT
        person.person_id::text AS "participantId",
        person.display_name AS "displayName",
        team.team_id::text AS "teamId",
        team.name AS "teamName",
        fixture_squad.role
      FROM fixture_squad
      INNER JOIN person
        ON person.person_id = fixture_squad.person_id
      INNER JOIN team
        ON team.team_id = fixture_squad.team_id
      WHERE fixture_squad.fixture_id = $1
      ORDER BY
        team.name ASC,
        person.display_name ASC,
        person.person_id ASC
    `,
    [fixtureId],
  );
};

```

```
    return result.rows;
}
```

This is important architecturally:

```
API concept: participant
      ↓
repository translates
      ↓
DB: person + fixture_squad + team
```

We're not inventing a competing schema.

5. Format and test what we have so far.

Run:

```
npx.cmd prettier --write apps/backend/src/database
apps/backend/src/modules/competitions apps/backend/src/modules/fixtures
apps/backend/src/modules/participants
```

Then:

```
npm.cmd run typecheck --workspace=@sport-analytics/backend
```

Then:

```
npm.cmd run lint --workspace=@sport-analytics/backend
```

If either command fails, **stop there and send me the error** rather than continuing blindly.

If both pass, continue.

6. Step 10: implement proper database cleanup when the backend shuts down.

Your existing:

```
apps/backend/src/index.ts
```

only closes the HTTP server.

Replace the whole file with:

```
import 'dotenv/config';

import { createApp } from './app';
import { loadEnvironment } from './config/env';
import { closeDatabasePool } from './database';

const environment = loadEnvironment();
const app = createApp({ environment });

const server = app.listen(environment.PORT, () => {
  console.log(`Sport Analytics API listening on port ${environment.PORT}`);
});

let shuttingDown = false;

function closeHttpServer(): Promise<void> {
  return new Promise((resolve, reject) => {
    server.close((error) => {
      if (error) {
        reject(error);
        return;
      }
      resolve();
    });
  });
}

async function shutdown(signal: string): Promise<void> {
  if (shuttingDown) {
    return;
  }

  shuttingDown = true;

  console.log(`${signal} received; shutting down`);

  try {
```

```

    await closeHttpServer();
  } catch {
    console.error('Failed to close HTTP server cleanly.');
```

```

    process.exitCode = 1;
  }

  try {
    await closeDatabasePool();
  } catch {
    console.error('Failed to close database pool cleanly.');
```

```

    process.exitCode = 1;
  }
}

process.on('SIGINT', () => {
  void shutdown('SIGINT');
});

process.on('SIGTERM', () => {
  void shutdown('SIGTERM');
});

```

This satisfies the cleanup requirement without leaking a raw database exception into logs.

Again, this does **not** connect through a Supabase data API. `closeDatabasePool()` is closing our `pg` PostgreSQL pool.

7. Step 11: change the existing integration test so it tests OUR transaction helper.

Open:

```
apps/backend/tests/database/transaction.database.test.ts
```

Delete its current contents.

Replace them with:

```

import { Pool } from 'pg';
import { afterAll, beforeAll, describe, expect, test } from 'vitest';

```

```

import { executeQuery, withTransaction } from '../../../src/database';
import { assertSafeTestDatabase } from '../../../scripts/test-database-safety';

describe('database transaction helper', () => {
  let pool: Pool | undefined;

  function getPool(): Pool {
    if (!pool) {
      throw new Error('Test database pool has not been initialised.');
    }

    return pool;
  }

  beforeAll(() => {
    const databaseUrl = assertSafeTestDatabase(
      process.env.DATABASE_URL_TEST,
      process.env.DATABASE_URL,
      process.env.NODE_ENV,
    );

    pool = new Pool({
      connectionString: databaseUrl.toString(),
    });
  });

  afterAll(async () => {
    if (pool) {
      await pool.end();
    }
  });

  test('commits a successful multi-step transaction', async () => {
    const databasePool = getPool();

    const originalResult = await executeQuery<{ note: string }>(
      databasePool,
      `
        SELECT note
        FROM database_health
      `
    );
  });
});

```

```

        WHERE id = 1
    `,
);

expect(originalResult.rowCount).toBe(1);

const originalNote = originalResult.rows[0].note;
const committedNote = 'Temporary transaction commit test value.';

try {
    const transactionResult = await withTransaction(databasePool, async
(client) => {
        await executeQuery(
            client,
            `
                UPDATE database_health
                SET note = $1
                WHERE id = 1
            `,
            [committedNote],
        );

        const insideTransaction = await executeQuery<{ note: string }>(
            client,
            `
                SELECT note
                FROM database_health
                WHERE id = 1
            `,
        );

        return insideTransaction.rows[0].note;
    });

    expect(transactionResult).toBe(committedNote);

    const afterCommit = await executeQuery<{ note: string }>(
        databasePool,
        `
            SELECT note
            FROM database_health

```

```

        WHERE id = 1
    `
);

expect(afterCommit.rows[0].note).toBe(committedNote);
} finally {
    await executeQuery(
        databasePool,
        `
            UPDATE database_health
            SET note = $1
            WHERE id = 1
        `
        ,
        [originalNote],
    );
}
});

test('rolls back all changes when an operation fails', async () => {
    const databasePool = getPool();

    const originalResult = await executeQuery<{ note: string }>(
        databasePool,
        `
            SELECT note
            FROM database_health
            WHERE id = 1
        `
    );

    expect(originalResult.rowCount).toBe(1);

    const originalNote = originalResult.rows[0].note;
    const temporaryNote = 'Temporary transaction rollback test value.';

    await expect(
        withTransaction(databasePool, async (client) => {
            await executeQuery(
                client,
                `
                    UPDATE database_health

```

```

        SET note = $1
        WHERE id = 1
    `,
    [temporaryNote],
);

const insideTransaction = await executeQuery<{ note: string }>(
    client,
    `
        SELECT note
        FROM database_health
        WHERE id = 1
    `,
);

expect(insideTransaction.rows[0].note).toBe(temporaryNote);

    throw new Error('Force transaction rollback.');
```

}},

).rejects.toThrow('Force transaction rollback.');

```

const afterRollback = await executeQuery<{ note: string }>(
    databasePool,
    `
        SELECT note
        FROM database_health
        WHERE id = 1
    `,
);

expect(afterRollback.rows[0].note).toBe(originalNote);
});
});
```

This is much better evidence than the old test because it now proves:

OUR withTransaction()

↓

commit works 

rollback works 

rather than merely proving PostgreSQL itself supports transactions.

8. Add a small unit test for safe error translation.

Create:

```
apps/backend/tests/unit/database-errors.test.ts
```

Paste:

```
import { describe, expect, test } from 'vitest';

import { DatabaseAccessError, translateDatabaseError } from
'../../src/database';

function postgresError(code: string, message: string): Error & { code: string }
{
  return Object.assign(new Error(message), { code });
}

describe('database error translation', () => {
  test('translates unique constraint failures without exposing raw details', ()
=> {
    const rawError = postgresError(
      '23505',
      'duplicate key value violates unique constraint competition_name_key',
    );

    const translated = translateDatabaseError(rawError);

    expect(translated).toBeInstanceOf(DatabaseAccessError);
    expect(translated.code).toBe('DATABASE_CONFLICT');
    expect(translated.message).toBe(
      'The database operation conflicts with existing data.',
    );
    expect(translated.message).not.toContain('competition_name_key');
  });

  test('translates foreign-key failures', () => {
    const translated = translateDatabaseError(
      postgresError('23503', 'raw foreign key database detail'),
    );
  });
});
```

```

);

expect(translated.code).toBe('DATABASE_REFERENCE_ERROR');
expect(translated.message).toBe(
  'The database operation references unavailable data.',
);
});

test('translates connection failures', () => {
  const translated = translateDatabaseError(
    postgresError('08006', 'raw database connection failure'),
  );

  expect(translated.code).toBe('DATABASE_UNAVAILABLE');
  expect(translated.message).toBe('The database is currently unavailable.');
```

```

});

test('uses a safe fallback for unknown database failures', () => {
  const translated = translateDatabaseError(
    postgresError('XX000', 'internal PostgreSQL implementation detail'),
  );

  expect(translated.code).toBe('DATABASE_OPERATION_FAILED');
  expect(translated.message).toBe(
    'The database operation could not be completed.',
  );
  expect(translated.message).not.toContain('PostgreSQL');
});
});

```

This directly gives you test evidence for:

Errors are translated consistently without exposing credentials or raw sensitive details.

9. Format and run the unit/backend checks again.

```

npx.cmd prettier --write apps/backend/src
apps/backend/tests/database/transaction.database.test.ts

```

```
apps/backend/tests/unit/database-errors.test.ts
```

Then:

```
npm.cmd run test:unit
```

Then:

```
npm.cmd run typecheck --workspace=@sport-analytics/backend  
npm.cmd run lint --workspace=@sport-analytics/backend  
npm.cmd run build --workspace=@sport-analytics/backend
```

All should pass before you go further.

l0. **Step 12: run the actual database integration test.**

First reset the dedicated test database:

```
npm.cmd run db:test:reset --workspace=@sport-analytics/backend
```

Then seed it:

```
npm.cmd run db:test:seed --workspace=@sport-analytics/backend
```

Then:

```
npm.cmd run test:database
```

You specifically want the output to show both tests passing:

```
commits a successful multi-step transaction  
rolls back all changes when an operation fails
```

That output is your #62 integration-test evidence.

Do not run these tests against your normal development database. The existing `assertSafeTestDatabase()` guard is deliberately there to prevent that.

l1. **Step 13: add the database-access documentation.**

Create:

```
docs/database/access.md
```

Paste:

```
# Database access and transactions
```

```
## Application database boundary
```

Application data is accessed through the handwritten backend and its repository layer.

The application-data path is:

```
```text
```

```
Frontend or API consumer
```

```
 ↓ HTTP
```

```
Handwritten Express API
```

```
 ↓
```

```
Service
```

```
 ↓
```

```
Repository
```

```
 ↓
```

```
pg
```

```
 ↓
```

```
PostgreSQL
```

```
```
```

The frontend does not access PostgreSQL directly and does not use generated Supabase Data API endpoints.

```
## Connection and configuration
```

The application uses the standard PostgreSQL `pg` driver.

The accepted connection approach remains the one recorded in ADR-003:

- `DATABASE\_URL` is the application database connection-string source;
- the hosted development database is PostgreSQL on Supabase;
- the backend connects through the Supervisor session-mode pooler;

- TLS certificate verification remains enabled;
- the committed CA certificate is used from `apps/backend/certs/supabase-ca.crt`;
- generated PostgREST/Data API endpoints are not used for application data.

The reusable application pool is implemented in:

```
```text
apps/backend/src/database/pool.ts
```
```

Pool creation is lazy so importing a module does not itself establish a database connection.

Database-access boundaries

SQL used by application features belongs behind repository boundaries.

The initial repository boundaries are:

```
Application area	Database source
Competition	`competition`
Fixture	`fixture`
Fixture participant	`person`, `fixture_squad`, `team`
```

The participant repository maps the existing sport-domain schema to an application participant record. It does not introduce or require a separate `participant` table.

Controllers must remain thin and must not contain SQL.

Services may coordinate repositories and transaction boundaries, but direct application queries remain in repositories.

Query execution

Repositories use the shared query executor:

```
```text
apps/backend/src/database/query.ts
```

```
```
```

Queries use PostgreSQL parameters such as:

```
```sql
WHERE fixture_id = $1
```
```

instead of interpolating input into SQL strings.

The query helper translates PostgreSQL failures into safe application database errors before they can cross the database-access boundary.

Identifiers

PostgreSQL `bigint` identifiers are converted to strings when read into application records.

For example:

```
```sql
fixture_id::text AS "fixtureId"
```
```

This follows the shared API identifier convention and avoids unsafe JavaScript number conversion.

Transactions

Atomic multi-step operations use:

```
```text
withTransaction()
```
```

from:

```
```text
apps/backend/src/database/transaction.ts
```
```

The helper:

1. checks out a client from the shared pool;
2. begins a transaction;
3. runs all supplied operations using the same client;
4. commits when all operations succeed;
5. rolls back when an operation fails; and
6. releases the checked-out client in all cases.

A service can therefore coordinate multiple repository operations without placing transaction logic inside individual repositories.

Conceptually:

```
```text
BEGIN
 repository operation A
 repository operation B
 repository operation C
COMMIT
```
```

If any operation fails:

```
```text
BEGIN
 repository operation A
 repository operation B
 failure
ROLLBACK
```
```

This supports the project requirement that multi-record acceptance is atomic: all related records are persisted or none are.

Error handling

Raw PostgreSQL errors are translated into stable application-level database errors.

Current categories include:

```
```text
```

```
DATABASE_UNAVAILABLE
DATABASE_CONFLICT
DATABASE_REFERENCE_ERROR
DATABASE_CONSTRAINT_ERROR
DATABASE_OPERATION_FAILED
DATABASE_TRANSACTION_FAILED
```
```

Public API code must not expose raw PostgreSQL messages, connection strings, credentials, SQL fragments or internal driver details.

Feature-specific services and API middleware may translate these database errors further into the shared API error contract where appropriate.

Connection cleanup

The application closes the shared PostgreSQL pool during graceful shutdown after it stops accepting new HTTP requests.

Checked-out transaction clients are released in a `finally` block so successful and failed operations both return their connection to the pool.

Testing

Unit tests verify database error translation.

Database integration tests verify the reusable transaction helper against the isolated test database.

Run:

```
```text
```

```
npm run db:test:reset --workspace=@sport-analytics/backend
npm run db:test:seed --workspace=@sport-analytics/backend
npm run test:database
```
```

The transaction integration tests cover:

- successful commit; and
- rollback after a failed operation.

Scope

This database foundation does not implement public HTTP endpoints.

Competition, fixture and participant HTTP reads are implemented separately through the handwritten API and consume these repository boundaries.

AI Declaration

The preceding document was planned, generated, reviewed and edited with the assistance of ChatGPT-Web[GPT-5.6 Sol].

This explicitly preserves all the decisions already made instead of inventing a new database architecture.

L2. Add that page to MkDocs.

Open:

```
mkdocs.yml
```

Find the existing database line, probably currently similar to:

- Database: database/overview.md

Replace only that part with:

- Database:
 - Overview: database/overview.md
 - Access and Transactions: database/access.md

Leave your #58 API documentation entries unchanged.

L3. Build the docs.

Run:

```
python -m mkdocs build --strict
```

If it passes, great.

Then check:

```
git status --short site
```

If MkDocs has created/generated `site/` changes, remove them exactly as we did before:

```
git restore --source=HEAD --staged --worktree -- site
git clean -fd -- site
```

Then:

```
git status --short site
```

You want no output.

L4. **Run the full #62 verification.**

Run:

```
npx.cmd prettier --check apps/backend/src/database
apps/backend/src/modules/competitions apps/backend/src/modules/fixtures
apps/backend/src/modules/participants apps/backend/src/index.ts
apps/backend/tests/database/transaction.database.test.ts
apps/backend/tests/unit/database-errors.test.ts docs/database/access.md
mkdocs.yml
```

Then:

```
npm.cmd run test:unit
npm.cmd run test:database
npm.cmd run typecheck --workspace=@sport-analytics/backend
npm.cmd run lint --workspace=@sport-analytics/backend
npm.cmd run build --workspace=@sport-analytics/backend
python -m mkdocs build --strict
```

Then, because the methodology specifically says to run it:

```
npm.cmd run check
```

If `npm run check` fails, **do not immediately modify unrelated files**. Send me the failure. We need to determine whether it is your #62 change or something inherited from `main`.

Finally:

```
git diff --check  
git status --short
```

L5. AI evidence: follow the CURRENT repository convention, not the stale path in the issue description.

Your #62 issue says:

```
evidence/ai/ai-usage-register.csv
```

but #58 already moved your current repository to the per-member register convention.

So use:

```
evidence/ai/registers/shayna-underslak.csv
```

Do **not** recreate the old shared register.

Save/export the #62 AI transcript into your transcript folder. We'll make the exact CSV row once the implementation/tests are complete so we can accurately state what you verified.

Your README should already contain:

```
ChatGPT-Web[GPT-5.6 Sol]
```

from #58, so **do not add it a second time**.

L6. Use two commits, as the methodology asks for small meaningful commits.

Your first commit should contain the actual foundation and tests:

```
apps/backend/src/database/errors.ts
apps/backend/src/database/index.ts
apps/backend/src/database/pool.ts
apps/backend/src/database/query.ts
apps/backend/src/database/transaction.ts
apps/backend/src/index.ts
apps/backend/tests/database/transaction.database.test.ts
apps/backend/tests/unit/database-errors.test.ts
```

Stage those only:

```
git add apps/backend/src/database
git add apps/backend/src/index.ts
git add apps/backend/tests/database/transaction.database.test.ts
git add apps/backend/tests/unit/database-errors.test.ts
```

Check:

```
git diff --cached
```

Then commit:

```
git commit -m "feat(database): add pool and transaction foundation" -m "Refs #62" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Sol]"
```

The second commit will contain:

```
competition.repository.ts
fixture.repository.ts
participant.repository.ts
docs/database/access.md
mkdocs.yml
AI evidence/register
```

and we'll use something like:

```
feat(database): add initial repository boundaries
```

with the same `Refs #62` and `Assisted-by` trailer.

But **don't make the second commit yet**. First get Steps 1–14 passing, then we'll do your AI evidence and final second commit properly.

Where this leaves us for #47

After #62:

```
#58 ✅ Shared API contracts
      ↓
#62 ✅ DB/repository foundation
      ↓
#47
```

Then #47 can cleanly do:

```
GET /api/v1/competitions
      ↓
competition service
      ↓
competition repository

GET /api/v1/fixtures/:id
      ↓
fixture service
      ↓
fixture repository

GET /api/v1/fixtures/:id/participants
      ↓
participant service
      ↓
participant repository
```

So everything we're adding now is directly preparing for #47, while **not prematurely implementing #47 inside #62**.

For now, do through **Step 5 first** — fill the three repositories, fix `transaction.ts`, then run the backend `typecheck` and `lint`. If either gives an error, paste it before moving on.

8/9/2026, 3:26:22 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run db\:test\:reset --workspace=@sport-analytics/backend

> @sport-analytics/backend\@0.1.0 db\:test\:reset
> cross-env NODE_ENV=test tsx --env-file=.env.test scripts/reset-test-database.ts

node: .env.test: not found
npm error Lifecycle script `db:test:reset` failed with error:
npm error code 9
npm error path
C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool\apps\backend
npm error workspace @sport-analytics/backend\@0.1.0
npm error location
C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool\apps\backend
npm error command failed
npm error command C:\Windows\system32\cmd.exe /d /s /c cross-env NODE_ENV=test tsx --env-file=.env.test scripts/reset-test-database.ts
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/9/2026, 3:38:06 PM



Thinking



Inspecting Repository Environment Configuration

- **Setting up test configuration**

I'm identifying the immediate issue: `apps/backend/.env.test` is missing. I'll create it with separate development and test database URLs, ensuring the test database name includes `test` and is isolated before running migrations or resets.

- **Set up test configuration**



Inspecting Test Database Configuration

- **Inspected repository, test environment, and database configurations**

✓ Worked for 34s

That error is expected: the repo's test scripts are explicitly trying to load:

```
apps/backend/.env.test
```

and you don't have that file yet.

Do **not** point this at your normal Supabase development database. The reset script literally does:

```
DROP SCHEMA public CASCADE;  
CREATE SCHEMA public;
```

so using the wrong database would wipe your development schema.

Do this exactly:

1. Go into the backend folder:

```
cd apps/backend
```

2. Create the test env file from the existing example:

```
Copy-Item .env.example .env.test
```

3. Open:

```
apps/backend/.env.test
```

For the database section, you need a **dedicated test database**.

If you have local PostgreSQL running, use:



```
NODE_ENV=test
```

```
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/sport_analytics
```

```
DATABASE_URL_TEST=postgresql://postgres:postgres@localhost:5432/sport_analytics_test
```

The critical part is:

```
sport_analytics_test
```

The repo's own safety code requires the test database name to contain the word:

```
test
```

and it also rejects `DATABASE_URL_TEST` if it matches `DATABASE_URL`.

4. If that local test database does not exist yet, create it.

If you have PostgreSQL CLI available:

```
createdb -U postgres sport_analytics_test
```

If that command is unavailable but you use pgAdmin, create a database named exactly:

```
sport_analytics_test
```

Do **not** put real credentials into the chat.

5. Confirm `.env.test` will not be committed.

Your `.gitignore` already contains:

```
.env.*  
!.env.example
```

so `.env.test` should be ignored.

Check:

```
git status --short
```

You should **not** see:

```
apps/backend/.env.test
```

6. Now return to the repo root:

```
cd ../../
```

Then rerun:

```
npm.cmd run db:test:reset --workspace=@sport-analytics/backend
```

If successful, it should:

```
connect to sport_analytics_test
↓
drop/reset its public schema
↓
run all migrations
```

Then:

```
npm.cmd run db:test:seed --workspace=@sport-analytics/backend
```

Then:

```
npm.cmd run test:database
```

The two important #62 tests should be:

- ✓ commits a successful multi-step transaction
- ✓ rolls back all changes when an operation fails

One important point if you only have Supabase

If you **do not have local PostgreSQL** and your only database is the shared Supabase database, **stop before creating** `.env.test` **with that URL.**

Your current Supabase database is not safe for `db:test:reset`, because the reset script intentionally destroys the `public` schema. We would need to set up a genuinely isolated test database first.

So your next action is:

```
cd apps/backend  
Copy-Item .env.example .env.test
```

Then tell me one thing: **do you have PostgreSQL running locally, or have you only been using the Supabase-hosted database?** That determines the safe value for `DATABASE_URL_TEST`.

8/9/2026, 3:38:08 PM



im confused with what im doing in the .env.test.. where am i putting what and what does it mean to be runing postrge

8/9/2026, 3:40:56 PM



Thinking



Inspecting PostgreSQL Configuration and Test Safety

- Inspected PostgreSQL configuration, archive paths, and project files

✓ Worked for 33s

You're at the point where **you should not guess the** `.env.test` **values**. The file is just a set of environment variables telling the test scripts **which database they are allowed to destroy/reset**.

Your repo's `.env.example` already shows the intended shape:

```
NODE_ENV=development
PORT=3000
CORS_ORIGINS=http://localhost:5173

DATABASE_URL=postgresql://postgres:postgres@localhost:5432/sport_analytics

SUPABASE_URL=https://your-project-ref.supabase.co
SUPABASE_PUBLISHABLE_KEY=your-supabase-publishable-key

EXTERNAL_API_KEY=

API_VERSION=v1
CORS_ALLOWED_ORIGINS=http://localhost:5173
LOG_LEVEL=debug

DATABASE_URL_TEST=postgresql://postgres:postgres@localhost:5432/sport_analytics_t
```

For `.env.test`, the important parts are really just these:

```
NODE_ENV=test
```

```
DATABASE_URL=YOUR_NORMAL_DEVELOPMENT_DATABASE_URL
```

```
DATABASE_URL_TEST=YOUR_SEPARATE_TEST_DATABASE_URL
```

The reason both exist is safety:

- `DATABASE_URL` = your normal development database
- `DATABASE_URL_TEST` = a completely separate disposable database used only by tests

Your repository checks that they are **not the same** and that the test database name contains the word `test`.

What does “running PostgreSQL locally” mean?

PostgreSQL is the actual database software.

You currently have two possible ways of using PostgreSQL:

Option A – Supabase-hosted PostgreSQL

Your computer

↓ internet

Supabase

↓

PostgreSQL database

or:

Option B – local PostgreSQL

Your computer

↓

PostgreSQL installed directly on your computer

↓

sport_analytics database

When I said “are you running PostgreSQL locally?”, I meant:

Have you installed PostgreSQL on your Windows computer itself?

For example, if you had installed PostgreSQL locally, you might have a program such as:

- PostgreSQL 16/17
- pgAdmin
- `psql`
- a Windows service called `postgresql-x64-...`

and this URL would work:

```
postgresql://postgres:postgres@localhost:5432/sport_analytics
```

`localhost` means **your own computer**.

I suspect you are not using local PostgreSQL

From how your project has been set up so far, I suspect your group is using **Supabase-hosted PostgreSQL** for development.

If that's true, then **do not just paste** this:

```
DATABASE_URL_TEST=postgresql://postgres:postgres@localhost:5432/sport_analytics_test
```

because there probably isn't a PostgreSQL server at `localhost:5432`.

And definitely do **not** put your normal Supabase `DATABASE_URL` into `DATABASE_URL_TEST`.

The reset script runs:

```
DROP SCHEMA public CASCADE;  
CREATE SCHEMA public;
```

So if you accidentally point `DATABASE_URL_TEST` at your group's real database, it could wipe your project tables.

So right now, do this

Open:

```
apps/backend/.env.test
```

For now, make it:

```
NODE_ENV=test

DATABASE_URL=
DATABASE_URL_TEST=
```

You can leave the other variables out unless the tests specifically need them.

Do not fill those two URLs yet.

Then we need to establish which database setup you actually have.

On Windows, run:

```
psql --version
```

and:

```
Get-Service *postgres*
```

Send me the output of those two commands.

If `psql --version` says something like:

```
psql (PostgreSQL) 17.x
```

and `Get-Service` shows a PostgreSQL service, then you probably have PostgreSQL locally and I'll show you exactly how to create:

```
sport_analytics_test
```

If instead you get:

```
'psql' is not recognized...
```

and no PostgreSQL service appears, then you are almost certainly **not running PostgreSQL locally**, and we should choose a safe isolated test-database approach

rather than risk your Supabase development database.

8/9/2026, 3:40:57 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> psql --version
psql : The term 'psql' is not recognized as the name of
a cmdlet, function, script file, or operable program.
Check the spelling of
the name, or if a path was included, verify that the
path is correct and try again.
At line:1 char:1
```

```
- psql --version
-
...
+ CategoryInfo          : ObjectNotFound: (psql:String)
[], CommandNotFoundException
+ FullyQualifiedErrorId :
CommandNotFoundException
...
```

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/9/2026, 3:43:48 PM



Thinking



Inspecting repository database setup documentation

- **Clarifying database setup**

I'm sorting out the architecture conflict: authentication can use the newer decision, while database access remains PostgreSQL. Since `psql` is unavailable, I'm checking whether PostgreSQL or Docker is already installed before recommending the simplest test-database setup.

- **Clarified database setup**

- **Inspected repository database and testing configuration documentation**



✓ Worked for 1m 56s

That output only tells us `psql` **isn't available on your command line**. It does **not yet prove PostgreSQL itself isn't installed**.

Also, don't put any real database URL into `.env.test` yet. The test reset command deliberately drops the whole `public` schema, so we want a completely disposable database.

For #62, `.env.test` can eventually be as simple as:

```
NODE_ENV=test
DATABASE_URL_TEST=postgresql://postgres:postgres@localhost:5433/sport_analytics_test
```

`DATABASE_URL_TEST` means: **“this database exists only for automated tests; the scripts are allowed to reset/delete its contents.”**

Your normal shared Supabase database must **never** go there.

What “running PostgreSQL” means

Your app uses PostgreSQL as its database. Currently your group's normal database is PostgreSQL hosted by Supabase.

For integration tests, we need a second PostgreSQL database that lives separately, ideally on your own computer:

```
Normal development
Backend → Supabase PostgreSQL
      ↑
    KEEP SAFE

Automated tests
Backend tests → disposable PostgreSQL
      ↑
    CAN RESET
```

You don't need `psql` specifically. Node's `pg` package can connect directly to PostgreSQL. We just need somewhere for that PostgreSQL test database to run.

Do these two checks now

Run:

```
Get-Service *postgres*
```

and then:

```
docker --version
```

Send me both outputs.

What happens next depends on them:

- If `Get-Service *postgres*` shows a PostgreSQL service, you probably already have local PostgreSQL installed.
- If `docker --version` works, we can create a **safe disposable PostgreSQL test database in Docker** very easily, without changing your Supabase database.
- If neither exists, we'll set up one local test option. We will **not** use your real Supabase database as `DATABASE_URL_TEST`.

For now, leave `.env.test` alone or keep only:

```
NODE_ENV=test
```

and send me the results of those two commands.

8/9/2026, 3:43:49 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Service *postgres*
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> docker --version
docker : The term 'docker' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was included, verify that the path is correct and try again.
At line:1 char:1

- docker --version
-
...
+ CategoryInfo          : ObjectNotFound:
(docker:String) [], CommandNotFoundException
+ FullyQualifiedErrorId :
CommandNotFoundException
...
```

8/9/2026, 3:47:34 PM



Thinking

- **Choosing a database path**

I'm narrowing this to the simplest setup: install PostgreSQL locally, then create the test database and run the existing integration tests. I'm avoiding Docker and generated APIs because neither is currently available.

- **Chose a database path**



Searching the web

postgresql.org

enterprisedb.com

wiki.postgresql.org

odbc.postgresql.org

- **Searched 4 websites**



Inspecting Project PostgreSQL Configuration and Safety

- **Installing PostgreSQL locally**



I'm recommending native PostgreSQL on Windows as the least disruptive path. First run the existing database check to identify the development server's major version, then install that matching version locally and create the isolated test database.

- **Installed PostgreSQL locally**



Inspecting Test Database Reset and Seeding Scripts

- **Inspected PostgreSQL configuration, setup documentation, and database reset scripts**

✓ Worked for 45s

That tells us something useful: **you currently do not have a local PostgreSQL server or Docker available.**

So don't keep fighting `.env.test` yet. The safest route is to install **plain PostgreSQL locally just for the disposable test database**. That does **not** contradict your project docs: your normal development DB stays Supabase-hosted, while the local DB exists only for integration tests. Your own setup docs already describe the hosted PostgreSQL development DB and treat local infrastructure as optional. The official PostgreSQL Windows page provides a Windows installer, maintained/certified through EDB. ([PostgreSQL](#))

1. First, find your project's PostgreSQL major version

From the repo root run:

```
npm.cmd run db:check --workspace=@sport-analytics/backend
```

Your existing project documentation says this command reports the database name and server version.

Do not send me the connection string or password. Just send me the line that says the PostgreSQL/server version.

We'll ideally install the same major version locally instead of guessing.

2. Then install PostgreSQL on Windows

Use the Windows installer from the official PostgreSQL download page. PostgreSQL recommends the binary Windows installer for normal Windows users rather than

building it yourself. ([PostgreSQL](#))

During the installer:

- install **PostgreSQL Server**;
- install **Command Line Tools**;
- pgAdmin is useful but optional;
- keep port `5432` unless the installer says it is already occupied;
- choose a password for the local `postgres` user;
- **remember that password**;
- default locale is fine;
- you can skip Stack Builder afterwards.

This PostgreSQL installation is **only on your laptop**.

Your architecture then becomes:

NORMAL DEVELOPMENT

backend

↓

Supabase-hosted PostgreSQL

↓

real shared dev data

INTEGRATION TESTING

backend tests

↓

PostgreSQL on Shayna's laptop

↓

sport_analytics_test

↓

safe to wipe/reset

3. After installation, check it

Close PowerShell, open a **new** PowerShell window and run:

```
psql --version
```

You should now get something like:

```
psql (PostgreSQL) 17.x
```

Also:

```
Get-Service *postgres*
```

This time you should see a running PostgreSQL service.

4. Create the disposable database

Once PostgreSQL is installed, open **SQL Shell (psql)** from the Windows Start menu.

It will ask things like:

```
Server [localhost]:  
Database [postgres]:  
Port [5432]:  
Username [postgres]:  
Password for user postgres:
```

For the first four, you can normally press Enter to accept the defaults.

For the password, type the password you selected during installation.

Then you'll see:

```
postgres=#
```

Run:

```
CREATE DATABASE sport_analytics_test;
```

It should respond:

```
CREATE DATABASE
```

Then:

```
\q
```

That exits.

5. Now `.env.test` finally makes sense

The file should be:

```
apps/backend/.env.test
```

And for these database tests, it can be very small:

```
NODE_ENV=test
DATABASE_URL_TEST=postgresql://postgres:YOUR_LOCAL_PASSWORD@localhost:5432/sport_
```

Replace:

```
YOUR_LOCAL_PASSWORD
```

with the PostgreSQL password **you just created on your own computer**.

For example, if your local password were hypothetically `abc123`:

```
NODE_ENV=test
DATABASE_URL_TEST=postgresql://postgres:abc123@localhost:5432/sport_analytics_test
```

Don't use that example password.

And importantly, **you do not need to put your real Supabase** `DATABASE_URL` in `.env.test`. Your safety script allows `DATABASE_URL` to be absent; it mainly requires that `DATABASE_URL_TEST` exists, is PostgreSQL, and that the database name contains `test`.

Your repo checks exactly that:

```
NODE_ENV must equal test
DATABASE_URL_TEST must exist
database name must contain "test"
```

So:

| | |
|----------------------|---|
| sport_analytics_test | ✓ |
| sport_analytics | ✗ |
| postgres | ✗ |

6. Why this database is safe to reset

Your command:

```
npm.cmd run db:test:reset --workspace=@sport-analytics/backend
```

will connect only to:

```
sport_analytics_test
```

and run:

```
DROP SCHEMA public CASCADE;  
CREATE SCHEMA public;
```

Then it reapplies your committed migrations.

So you can repeatedly do:

```
wipe test DB  
↓  
recreate schema  
↓  
run migrations  
↓  
seed  
↓  
test
```

without touching the shared Supabase development database.

7. Then run #62's tests

Once installation and `.env.test` are done:

```
npm.cmd run db:test:reset --workspace=@sport-analytics/backend
```

then:

```
npm.cmd run db:test:seed --workspace=@sport-analytics/backend
```

then:

```
npm.cmd run test:database
```

And for #62 we want both:

- ✓ commits a successful multi-step transaction
- ✓ rolls back all changes when an operation fails

For now, **don't install anything until you run:**

```
npm.cmd run db:check --workspace=@sport-analytics/backend
```

Send me only the server/PostgreSQL version it reports, and I'll tell you which PostgreSQL major version to install locally.

Sources

[PostgreSQL: Windows installers](https://www.postgresql.org/) [postgresql.org](https://www.postgresql.org/)

8/9/2026, 3:47:42 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run db\:check --workspace=@sport-analytics/backend

> @sport-analytics/backend\@0.1.0 db\:check
> tsx --env-file=.env scripts/check-connection.ts

node: .env: not found
npm error Lifecycle script `db\:check` failed with error:
npm error code 9
npm error path
C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool\apps\backend
npm error workspace @sport-analytics/backend\@0.1.0
npm error location
C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool\apps\backend
npm error command failed
npm error command C:\Windows\system32\cmd.exe /d /s /c tsx --env-file=.env scripts/check-connection.ts
```

8/9/2026, 3:50:11 PM



Thinking

- **Investigated database scripts, environment configuration, and check errors**
-  **Inspecting Latest Zip Contents Directly**
- **Clarifying database setup**

I'm correcting my earlier advice: running `db\:check` before creating `.env` will fail normally. I'll first have you copy `.env.example`, configure the required database connection, and only then rerun the check.
- **Clarified database setup**
- **Inspected zip contents and database configuration documentation**

✓ Worked for 47s

That error makes sense - and I sent you down an unnecessary detour with `db:check`.

`db:check` is checking the **normal shared development database**, and its script requires:

```
apps/backend/.env
```

You don't currently have that file. **We do not need to solve that in order to set up the isolated test database for #62.**

For #62, the important thing is:

```
apps/backend/.env.test
```

not `.env`.

What the two files mean

```
apps/backend/.env
    ↓
normal DEVELOPMENT database
    ↓
your group's shared Supabase PostgreSQL
    ↓
DO NOT wipe/reset
```

versus:

```
apps/backend/.env.test
    ↓
separate TEST database
    ↓
only automated tests
    ↓
SAFE to wipe/reset repeatedly
```

Your `db:test:reset` script deliberately runs:

```
DROP SCHEMA public CASCADE;  
CREATE SCHEMA public;
```

so **it absolutely must not point at your group's normal Supabase database.**

Where we actually are

From the checks you ran:

| | | |
|--------------------|---|---------------|
| psql | ✗ | not installed |
| PostgreSQL service | ✗ | not installed |
| Docker | ✗ | not installed |

So right now your computer has **nowhere to run the isolated PostgreSQL test database.**

That is the actual blocker.

This is not a problem with your #62 code.

What I recommend

For your laptop, install PostgreSQL locally. Then your setup will be:

NORMAL PROJECT DATA

Backend

↓

shared Supabase PostgreSQL

↓

real project data

#62 AUTOMATED TESTS

Backend tests

↓

PostgreSQL installed on your laptop

↓

sport_analytics_test

↓
disposable

This **does not change the project's chosen database architecture**. The production/development application still uses the hosted PostgreSQL setup. Local PostgreSQL is merely test infrastructure.

And it doesn't affect the handwritten-API rule either: the project explicitly requires your application API to be designed and implemented by the team.

project_briefs.pdf

You do NOT need `.env` right now

So don't create/fill:

apps/backend/.env

just to make `db:check` work.

Also don't put any Supabase password/URL into `.env.test`.

Your next step is simply installing PostgreSQL

Once installed, we'll create one database named:

sport_analytics_test

and then `.env.test` becomes very understandable:

```
NODE_ENV=test
DATABASE_URL_TEST=postgresql://postgres:YOUR_LOCAL_PASSWORD@localhost:5432/sport_
```

Breaking that URL down:

```
postgresql://
  ↓
  use PostgreSQL

postgres
  ↓
  local database username
```

YOUR_LOCAL_PASSWORD
↓
password YOU choose during installation

localhost
↓
your own laptop

5432
↓
default PostgreSQL port

sport_analytics_test
↓
the disposable database used by tests

Nothing in that URL is Supabase.

One correction to my previous instruction

We **do not need to know the Supabase server version before installing PostgreSQL locally** for these tests. The purpose here is to verify transaction behaviour and migrations, not reproduce Supabase byte-for-byte.

So don't keep trying this:

```
npm.cmd run db:check --workspace=@sport-analytics/backend
```

for now.

Your project methodology requires relevant tests and checks before the PR, but it doesn't require you to use the shared development database for integration testing.

`Git_Methodology.md`

Where #62 stands

You have not broken anything:

| | |
|-------------------------|---------------|
| #62 implementation code | ✅ progressing |
| Repository boundaries | ✅ |
| Transaction helper | ✅ |

| | |
|---------------------------|----------------|
| Error handling | ✓ |
| Integration tests written | ✓ |
| Test DB infrastructure | ← WE ARE HERE |
| Documentation | next |
| Verification | after DB setup |

And the final goal is still:

```
#58 ✓  
  ↓  
#62 ← current  
  ↓  
#47 public read API
```

Next, we install PostgreSQL locally. If you want, I can give you the exact Windows installation steps - what to download and **exactly what to tick/select on every installer screen.**

8/9/2026, 3:50:23 PM



yes pls

8/9/2026, 3:52:09 PM



Thinking



Searching the web

postgresql.org

en.wikipedia.org

play.google.com

freedownloadmanager.org

reddit.com

- Searched 5 websites

✓ Worked for 19s

Yes. Here is the exact route I'd use on your Windows laptop for **#62 test infrastructure only**.

As of now, PostgreSQL **18.4** is the current stable major release, while PostgreSQL 19 is still beta, so install **PostgreSQL 18**, not 19 Beta. The official PostgreSQL Windows download page points Windows users to the EDB interactive installer, which includes the server, pgAdmin and Stack Builder. ([PostgreSQL](https://www.postgresql.org))

1. Download PostgreSQL

Go to the official PostgreSQL Windows download page via the citation above and choose **Download the installer**.

On the EDB download page, select:

PostgreSQL version: 18.x

Operating system: Windows x86-64

If you see something like **18.4**, choose that.

Do **not** choose PostgreSQL 19 Beta.

Download the `.exe` installer.

2. Start the installer



Double-click the downloaded installer.

If Windows asks:

Do you want to allow this app to make changes to your device?

Choose:

Yes

The EDB installer is the installer linked by the official PostgreSQL project. ([PostgreSQL](#))

3. Welcome screen

Click:

Next

4. Installation Directory

You will probably see something similar to:

C:\Program Files\PostgreSQL\18

Leave it exactly as the default.

Click:

Next

5. Select Components

You should see options approximately like:

- ☒ PostgreSQL Server
- ☒ pgAdmin 4
- ☒ Stack Builder
- ☒ Command Line Tools

For you, keep:

- ☒ PostgreSQL Server
- ☒ pgAdmin 4
- ☒ Command Line Tools

You can also leave Stack Builder ticked, although we don't need it for this project. The command-line tools include things such as `psql`, which will be useful for creating and checking your test database. ([EDB](#))

So easiest is simply:

Leave all the default components selected.

Click:

Next

6. Data Directory

It will probably suggest something like:

C:\Program Files\PostgreSQL\18\data

Leave the default.

Click:

Next

7. Password - important

You'll now be asked to create a password for the PostgreSQL superuser:

postgres

This is **not your Supabase password**.

This is a new password only for PostgreSQL running on your laptop.

Choose a strong password you will remember.

For example, conceptually:

```
username: postgres
password: <your-new-local-password>
```

Don't send me the password.

We'll later use it locally in:

```
apps/backend/.env.test
```

The installer uses this password for the local PostgreSQL superuser. ([EDB](#))

Click:

Next

8. Port

You should see:

```
5432
```

Leave it as:

```
5432
```

That is the normal PostgreSQL port.

Since our earlier checks found no PostgreSQL service on your machine, it is unlikely to conflict with an existing local PostgreSQL server.

Click:

Next

9. Locale

You'll probably see:

```
[Default locale]
```

Leave:

Default locale

We don't need a special locale for these transaction tests.

Click:

Next

10. Pre-installation summary

You'll get a summary showing things such as:

PostgreSQL 18
Port 5432
Installation directory
Data directory

Click:

Next

Then click:

Next

again if it gives you a **Ready to Install** screen.

The installer will now install PostgreSQL. ([EDB](#))

11. Stack Builder at the end

At the end you may see:

☒ Launch Stack Builder at exit

You **do not need Stack Builder** for #62.

Untick:

☐ Launch Stack Builder at exit

and click:

Finish

Stack Builder is for optional additional PostgreSQL modules and drivers; it isn't necessary for our local transaction test database. ([EDB](#))

12. Now restart PowerShell

Close your current PowerShell window completely.

Open a **new PowerShell**.

Run:

```
psql --version
```

Hopefully you'll now see something similar to:

```
psql (PostgreSQL) 18.4
```

Then run:

```
Get-Service *postgres*
```

You should see a PostgreSQL service with a status similar to:

| Status | Name |
|---------|-------------------|
| ----- | ---- |
| Running | postgresql-x64-18 |

13. If `psql` still says "not recognized"

Don't panic. PostgreSQL can be installed correctly without its `bin` directory being added to Windows PATH.

Try:

```
& "C:\Program Files\PostgreSQL\18\bin\psql.exe" --version
```

If that returns:

```
psql (PostgreSQL) 18.x
```

then PostgreSQL is installed correctly.

We can sort the PATH out afterwards if necessary.

14. Create `sport_analytics_test`

Now open the Start menu and search for:

```
SQL Shell (psql)
```

Open it.

You'll get prompts similar to:

```
Server [localhost]:  
Database [postgres]:  
Port [5432]:  
Username [postgres]:  
Password for user postgres:
```

Do this:

```
Server [localhost]:      press Enter  
Database [postgres]:    press Enter  
Port [5432]:            press Enter  
Username [postgres]:    press Enter  
Password:              type the password you chose
```

When you type the password, **you may see no characters appearing on screen**. That's normal.

If successful you'll arrive at:

```
postgres=#
```

Now type:

```
CREATE DATABASE sport_analytics_test;
```

Press Enter.

You want:

```
CREATE DATABASE
```

Then verify it exists:

```
\1
```

You should see:

```
sport_analytics_test
```

in the database list.

Then exit:

```
\q
```

15. Now create your `.env.test`

Go back to your project:

```
C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool
```

Open:

```
apps/backend/.env.test
```

For the moment, it only needs:

```
NODE_ENV=test  
DATABASE_URL_TEST=postgresql://postgres:YOUR_PASSWORD@localhost:5432/sport_analyt:
```

Replace:

```
YOUR_PASSWORD
```

with the local PostgreSQL password you just chose.

Do not send that file to me.

And do **not** put your Supabase URL in this file.

If your password contains characters like:

```
@  
:  
/  
#  
%
```

tell me **only that it contains special characters**, not what the password is. We may need to URL-encode it.

16. Make sure Git ignores it

From the repo root:

```
git status --short
```

You should **not** see:

```
apps/backend/.env.test
```

because environment files containing credentials should remain ignored.

17. Then test the connection safely

Now run:

```
npm.cmd run db:test:reset --workspace=@sport-analytics/backend
```

This time the path should be:

```
your Node test script  
    ↓  
.env.test  
    ↓  
localhost:5432
```

↓
sport_analytics_test

NOT:

Supabase shared project DB ❌

Then:

```
npm.cmd run db:test:seed --workspace=@sport-analytics/backend
```

Then finally:

```
npm.cmd run test:database
```

For #62, we're aiming to see:

- ✓ commits a successful multi-step transaction
- ✓ rolls back all changes when an operation fails

So right now

Don't worry about Steps 14-17 yet.

Do **Steps 1-12** first: install PostgreSQL 18, restart PowerShell, then run:

```
psql --version  
Get-Service *postgres*
```

Send me those **two outputs only** and I'll take you through creating
sport_analytics_test safely.

Sources

[PostgreSQL: Windows installers](https://www.postgresql.org/) postgresql.org

[Installing PostgreSQL on Windows](https://enterprisedb.com/blog/installing-postgresql-on-windows/) enterprisedb.com

8/9/2026, 3:52:11 PM



do i need this?? i sthere no other/better way? why do our tests require this?

8/9/2026, 3:53:03 PM



Thinking

- **Planning integration tests**

I'm considering whether mocked transaction tests are sufficient, but the acceptance criteria require real commit and rollback coverage. A dedicated isolated PostgreSQL environment seems preferable, with safeguards ensuring tests cannot target production.

- **Planned integration tests**

✓ Worked for 14s

You **do need a real PostgreSQL database somewhere for the #62 integration tests**, but you **do not necessarily need to install PostgreSQL locally**. I pushed you too quickly toward a local install.

The reason is simply this acceptance criterion from #62:

Automated integration tests cover successful commit and rollback.

A unit test can mock this:

```
BEGIN
COMMIT
ROLLBACK
```

but that only proves our JavaScript called the right methods. It does **not** prove that the transaction helper actually works with PostgreSQL.

For #62, we want to prove:

```
real pg driver
↓
```

```
real PostgreSQL
  ↓
BEGIN
  ↓
write data
  ↓
COMMIT
  ↓
data really persists
```

and:

```
real PostgreSQL
  ↓
BEGIN
  ↓
write data
  ↓
failure
  ↓
ROLLBACK
  ↓
data really did NOT persist
```

That is why the repository has `test:database` and a separate `DATABASE_URL_TEST` concept in the first place.

But local PostgreSQL is only one option

You essentially have three choices:

| Option | Good? | My view |
|---|-------|--|
| Install PostgreSQL locally | ✓ | Works, but extra setup on every teammate's machine |
| Use a separate hosted PostgreSQL test database | ✓✓ | Probably better for your team |
| Mock PostgreSQL completely | ⚠ | Not enough for #62's integration-test requirement |

For **your project**, I actually prefer a **dedicated hosted test database** if your group is comfortable creating one.

Then your setup is:

DEVELOPMENT

Backend

↓

Supabase PostgreSQL project A

↓

shared development data

INTEGRATION TESTING

Automated tests

↓

separate PostgreSQL database/project B

↓

disposable test data

That means you don't install PostgreSQL or Docker at all.

And this **still obeys the hand-written API requirement**. Your test code connects using:

Node pg driver → PostgreSQL

not:

Supabase generated REST API

Why can't we just use the normal Supabase database?

Because your repository's reset tooling is intentionally destructive.

It can do things equivalent to:

```
DROP SCHEMA public CASCADE;  
CREATE SCHEMA public;
```

That is excellent for a disposable test database.

It is catastrophic for:

```
the group's normal Supabase development database
```

So this rule is important:

Integration tests must never be given credentials for a database containing project data that matters.

Could we skip `db:test:reset` ?

Technically, we could rewrite the testing strategy so the transaction test creates and cleans up only its own temporary objects.

But I **wouldn't do that for #62 without a good reason**, because your repository already has:

```
db:test:reset
db:test:seed
test:database
```

and existing safety infrastructure for an isolated test DB.

Changing that testing architecture just to avoid providing a test database would increase the scope of #62 and potentially contradict earlier testing work.

Could we use mocks?

We should use mocks/unit tests **as well**.

For example, a unit test can prove:

```
operation succeeds
→ COMMIT called
→ client.release called
```

and:

```
operation throws
→ ROLLBACK called
```

```
→ client.release called
```

Those are useful.

But #62 specifically asks for **integration tests**, so I wouldn't replace the real-Postgres test with mocks.

What I'd recommend now

Don't install PostgreSQL yet.

First ask your team whether there is already a **dedicated test PostgreSQL/Supabase database** from the testing/database work. Someone may already have created one for:

```
DATABASE_URL_TEST
```

That is especially worth checking because your repository already contains all of this test infrastructure — which strongly suggests whoever created it had an intended test-database setup.

Ask your team something like:

For the existing `db:test:reset`, `db:test:seed` and `test:database` scripts, do we already have a dedicated PostgreSQL test database / `DATABASE_URL_TEST` that the team is meant to use? I don't want to point the reset script at our normal Supabase development database.

If the answer is **yes**, great — use that.

If the answer is **no**, then we choose between:

- A. dedicated hosted test PostgreSQL
- or
- B. local PostgreSQL

For a six-person group, I'd lean toward **A**, because every developer and CI can test against the same isolated testing environment rather than everyone installing PostgreSQL individually.

So **don't install anything yet**. Check whether the team already has a `DATABASE_URL_TEST` setup first.

8/9/2026, 3:53:14 PM



do the docs not say anything about the tests anywhere? and how to run them or something?

8/9/2026, 3:56:23 PM



Thinking



Inspecting Repository Documentation and Test Scripts

- **Assessing test requirements**

I'm distinguishing routine local checks from database integration tests. The CI workflow provisions PostgreSQL automatically, while local setup lacks clear database instructions, so I'm weighing whether CI can validate the integration coverage before opening the PR.

- **Assessed test requirements**



Inspecting testing documentation and CI configuration

- **Clarifying test documentation**

I'm noticing the testing guide appears truncated, leaving commands without setup details. CI provides PostgreSQL 16, so I can run non-database checks locally, use a draft PR for integration verification, and restore clearer testing instructions in this change.

- **Clarified test documentation**

- **Inspected repository documentation, testing setup, database scripts, and CI configuration**

✓ Worked for 1m 10s

Yes - I checked the **latest repo itself**, and there's an important discovery:

You do not need to install PostgreSQL locally just to get #62 through the project's integration testing. I was pushing you further than the repository actually requires.

The project already has a complete disposable PostgreSQL test database configured in **Gitea CI**.

The current `.gitea/workflows/ci.yml` creates this automatically for every PR:

```
services:
  postgres:
    image: postgres:16
    env:
      POSTGRES_USER: test_user
      POSTGRES_PASSWORD: test_password
      POSTGRES_DB: sport_analytics_test
    ports:
      - 5432:5432
```

It then sets:

```
NODE_ENV: test
DATABASE_URL_TEST:
postgresql://test_user:test_password@localhost:5432/sport_analytics_test
```

and runs, in order:

```
npm ci

npm run db:test:reset --workspace=@sport-analytics/backend

npm run db:test:seed --workspace=@sport-analytics/backend

npm run check

npm run test:database

npm run test:e2e

npm run test:coverage
```

So the intended CI flow is literally:

```
PR opened
  ↓
Gitea runner starts PostgreSQL 16
  ↓
creates sport_analytics_test
```

```
↓
runs migrations
↓
seeds it
↓
runs DB integration tests
↓
throws the environment away afterwards
```

That is **much better than touching your shared Supabase database.**

Why were we confused, then?

Because there is actually a documentation problem in your repository.

Your current:

```
docs/development/testing.md
```

contains basically only:

```
npm ci

npm run db:test:reset --workspace=@sport-analytics/backend
npm run db:test:seed --workspace=@sport-analytics/backend

npm run test:unit
npm run test:frontend
npm run test:api
npm run test:contracts
npm run test:database
npm run test:e2e
npm run test:coverage
npm run check
```

It **doesn't explain where** `DATABASE_URL_TEST` **comes from or how the test database is provisioned locally.**

And I checked the Git history: that file previously had a much fuller testing strategy, but commit:

```
c67e6af
```

```
docs(testing): document automated testing foundation
```

actually reduced `docs/development/testing.md` from about **77 lines to 14 lines**.

So you weren't missing an obvious instruction. **The current documentation genuinely doesn't explain this properly.**

There is another clue

`apps/backend/.env.example` contains:

```
DATABASE_URL_TEST=postgresql://postgres:postgres@localhost:5432/sport_analytics_test
```

That demonstrates support for developers who **already have local PostgreSQL**, but the official setup guide does not say every developer must install PostgreSQL.

In fact `docs/development/setup.md` says:

```
A Docker-compatible runtime if using the optional local Supabase stack
```

So local infrastructure is explicitly described as **optional**.

What this means for you now

Do not install PostgreSQL.

Also do not create a fake `.env.test`, and definitely do not use your real Supabase database.

Continue #62 and let the **Gitea PR CI** perform the real PostgreSQL integration test.

Locally, run everything you can without the database:

```
npm.cmd run test:unit
npm.cmd run test:api
npm.cmd run typecheck --workspace=@sport-analytics/backend
npm.cmd run lint --workspace=@sport-analytics/backend
npm.cmd run build --workspace=@sport-analytics/backend
npm.cmd run check
```

For:

```
npm.cmd run test:database
```

your machine currently cannot run it because it has no local PostgreSQL test service. That's okay provided we record it truthfully and then verify it through the PR's CI PostgreSQL service.

And this actually affects #62 itself

One of #62's acceptance criteria is:

Backend documentation is updated

So while we're already creating `docs/database/access.md`, I think #62 should also make a **small, targeted improvement to** `docs/development/testing.md` explaining the database-test setup.

Something like:

Automated testing

Install the committed dependencies:

```
```bash
npm ci
```
```

Standard tests

Run:

```
```bash
npm run test:unit
npm run test:frontend
npm run test:api
npm run test:contracts
npm run test:e2e
npm run test:coverage
npm run check
```
```

Database integration tests

Database integration tests require a disposable PostgreSQL database.

They must never run against the shared development or production database.

The Gitea CI workflow automatically provisions an isolated PostgreSQL service with the database:

```
```text
sport_analytics_test
```
```

CI then runs:

```
```bash
npm run db:test:reset --workspace=@sport-analytics/backend
npm run db:test:seed --workspace=@sport-analytics/backend
npm run test:database
```
```

Developers who already have an isolated local PostgreSQL test database may run the same commands locally by providing an ignored:

```
```text
apps/backend/.env.test
```
```

containing a safe `DATABASE\_URL\_TEST`.

The repository's database safety guard requires:

- `NODE\_ENV=test`;
- `DATABASE\_URL\_TEST` to be configured;
- the test database name to contain `test`; and
- the test database URL not to equal the development database URL.

Do not point `DATABASE\_URL\_TEST` at the shared Supabase development database.

AI Declaration

The preceding document was reviewed and edited with the assistance of ChatGPT-Web[GPT-5.6 Sol].

That would actually **fix the confusion you just experienced**, while staying directly relevant to #62.

So our route is now much simpler:

```
#62 code
↓
unit/lint/typecheck/build locally
↓
docs
↓
commit/push
↓
Draft/PR
↓
Gitea CI creates PostgreSQL 16 automatically
↓
commit test ✓
rollback test ✓
↓
review
↓
merge
↓
#47 🎯
```

You can forget the PostgreSQL installation steps I gave you. The existing project CI already has the correct isolated test-database solution.

8/9/2026, 3:56:25 PM



ok, so whats my next xtep if those tests passed? also, our CI isnt working as the lecturer hasnt made a runner yet

8/9/2026, 3:59:35 PM



Thinking

- **Planning next steps**

Since the local tests are passing, I'm treating the missing CI runner as an infrastructure limitation rather than a code failure. I'm ready to update documentation, record local verification, commit the changes, and open the pull request with that limitation noted.

- **Planned next steps**

✓ Worked for 9s

Yes. If the backend unit/API/typecheck/lint/build checks you just ran all passed, **do not do more database setup**. Since your Gitea runner is unavailable for an external course-infrastructure reason, we'll document that honestly rather than pretending CI passed.

Your next steps are:

1. **Make the first small #62 code commit.**
2. **Finish the two documentation updates.**
3. **Run the documentation/local verification.**
4. **Update AI evidence.**
5. **Make the second commit.**
6. **Push and open the PR**, explicitly noting CI cannot execute because no runner has been provisioned.

1. First make the code/test commit

Before staging, run:

```
git status --short
```

For the first commit, stage only the database foundation, shutdown handling, and tests:

```
git add apps/backend/src/database
git add apps/backend/src/index.ts
git add apps/backend/tests/database/transaction.database.test.ts
git add apps/backend/tests/unit/database-errors.test.ts
```

Then check exactly what is staged:

```
git diff --cached --stat
git diff --cached
```

Do **not** include:

```
docs/
mkdocs.yml
AI evidence
site/
package-lock.json
```

in this first commit.

Commit:

```
git commit -m "feat(database): add pool and transaction foundation" -m "Refs #62" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Sol]"
```

That gives you a proper small, meaningful methodology-compliant commit.

2. Now finish `docs/database/access.md`

Create/open:

```
docs/database/access.md
```

Use the database-access documentation I gave you earlier.

The key things it must say are:

```
Frontend / API consumer
    ↓ HTTP
Handwritten Express API
    ↓
Service
    ↓
Repository
    ↓
pg
    ↓
PostgreSQL
```

and explicitly:

- application SQL belongs behind repositories;
- `DATABASE_URL` is the normal application connection configuration;
- the backend uses `pg`;
- no Supabase Data API/PostgREST is used as the application API;
- parameterised SQL is required;
- `withTransaction()` controls atomic multi-step work;
- checked-out clients are always released;
- PostgreSQL errors are translated before leaving the DB layer;
- PostgreSQL `bigint` IDs are converted to application strings;
- competition, fixture and participant repository boundaries now exist;
- participant maps the existing `person + fixture_squad + team` schema rather than inventing a new table.

At the bottom:

```
## AI Declaration
```

The preceding document was planned, generated, reviewed and edited with the assistance of ChatGPT-Web[GPT-5.6 Sol].

3. Fix `docs/development/testing.md`

This is worth doing in #62 because the current docs did not explain the exact problem you just ran into.

Keep the existing testing commands, but add a section like this:

`## Database integration tests`

Database integration tests require an isolated disposable PostgreSQL database.

They must never run against the shared development or production database.

The configured Gitea CI workflow provisions a PostgreSQL test service and runs:

```
```bash
npm run db:test:reset --workspace=@sport-analytics/backend
npm run db:test:seed --workspace=@sport-analytics/backend
npm run test:database
```
```

At present, execution of the CI workflow depends on a Gitea Actions runner being provisioned for the repository.

Developers who have access to an isolated local or hosted PostgreSQL test database may run the same tests locally using an ignored:

```
```text
apps/backend/.env.test
```
```

with a safe ``DATABASE_URL_TEST``.

The database safety checks require:

- ``NODE_ENV=test``;
- ``DATABASE_URL_TEST`` to be configured;
- the database name to contain ``test``; and
- the test database not to match the development database.

Never configure `DATABASE\_URL\_TEST` with the shared Supabase development database.

Then, because we're changing this document with AI assistance, its bottom should also include:

AI Declaration

The preceding document was reviewed and edited with the assistance of ChatGPT-Web[GPT-5.6 Sol].

Important wording difference: **the CI workflow is configured**, but it cannot currently execute because there is no runner. Don't write that CI automatically passes these tests today.

4. Add database docs to MkDocs

Open:

```
mkdocs.yml
```

Find the Database section.

Make it:

- Database:
 - Overview: database/overview.md
 - Access and Transactions: database/access.md

Don't disturb the API pages you added for #58.

5. Now verify the documentation

Format only your #62 files:

```
npx.cmd prettier --write apps/backend/src/database
apps/backend/src/modules/competitions apps/backend/src/modules/fixtures
apps/backend/src/modules/participants apps/backend/src/index.ts
apps/backend/tests/database/transaction.database.test.ts
```

```
apps/backend/tests/unit/database-errors.test.ts docs/database/access.md
docs/development/testing.md mkdocs.yml
```

Then:

```
python -m mkdocs build --strict
```

If that passes, remove generated site changes if they appear:

```
git restore --source=HEAD --staged --worktree -- site
git clean -fd -- site
```

Then:

```
git status --short site
```

You want **nothing**.

6. Run your local verification

Since the runner does not exist, your local evidence matters.

Run:

```
npm.cmd run test:unit
npm.cmd run test:api
npm.cmd run typecheck --workspace=@sport-analytics/backend
npm.cmd run lint --workspace=@sport-analytics/backend
npm.cmd run build --workspace=@sport-analytics/backend
```

Then run what the issue explicitly asks for:

```
npm.cmd run check
```

If `npm run check` passes, excellent.

If it still fails due to known unrelated formatting errors on `main`, **do not fix them inside #62**. Record the exact limitation.

What about `test:database` ?

If you do not have a disposable PostgreSQL test database locally, then:

```
npm.cmd run test:database
```

cannot currently be executed locally.

And because the lecturer has not provisioned the Gitea runner, CI cannot execute it either.

That means #62 has one temporarily **unverified acceptance criterion**:

Automated integration tests cover successful commit and rollback.

The tests themselves can exist, but we must not claim they passed against PostgreSQL until they actually have.

That does **not** mean you should invent a local PostgreSQL install just to hide the infrastructure problem.

Your PR should say that clearly.

7. Update your AI evidence

Save this conversation/transcript into:

```
evidence/ai/transcripts/shayna-underslak/
```

A sensible filename:

```
2026-08-09-issue-62-database-repositories.pdf
```

Check it contains no passwords, database URLs, API keys, or secrets.

Then add a row to:

```
evidence/ai/registers/shayna-underslak.csv
```

For the verification field, **do not say the PostgreSQL integration tests passed** unless you actually ran them.

Use wording along these lines:

Reviewed generated database foundation against existing ADRs, schema and handwritten API boundary; locally ran backend unit tests, API tests, lint, type-check, build and MkDocs strict build. Transaction integration tests were added for commit and rollback but could not be executed against the configured CI PostgreSQL service because the repository currently has no provisioned Gitea Actions runner.

That is accurate and defensible.

Your README should already list:

```
ChatGPT-Web[GPT-5.6 Sol]
```

from #58, so don't add it again.

8. Make your second commit

Now stage:

```
git add apps/backend/src/modules/competitions/competition.repository.ts
git add apps/backend/src/modules/fixtures/fixture.repository.ts
git add apps/backend/src/modules/participants/participant.repository.ts
git add docs/database/access.md
git add docs/development/testing.md
git add mkdocs.yml
git add evidence/ai/registers/shayna-underslak.csv
git add "evidence/ai/transcripts/shayna-underslak/2026-08-09-issue-62-database-repositories.pdf"
```

Check:

```
git diff --cached --stat
git diff --cached
```

Then:

```
git commit -m "feat(database): add initial repository boundaries" -m "Refs #62"
-m "Assisted-by: ChatGPT-Web[GPT-5.6 Sol]"
```

9. Final check before pushing

Run:

```
git status
git log --oneline -2
git diff main...HEAD --stat
git diff --check main...HEAD
```

You should have roughly:

```
commit 2 feat(database): add initial repository boundaries
commit 1 feat(database): add pool and transaction foundation
```

and a clean working tree.

Then:

```
git push -u origin feat/62-database-repositories
```

10. Open the PR, but be transparent about CI

Move #62 to:

In Review

when the PR opens.

In the PR's testing section, say something like:

Verification

Local verification completed:

- `npm run test:unit` ✓
- `npm run test:api` ✓
- backend type-check ✓
- backend lint ✓
- backend build ✓
- `python -m mkdocs build --strict` ✓
- `git diff --check` ✓

Database integration tests for successful commit and rollback are implemented.

The repository's Gitea workflow is configured to provision an isolated PostgreSQL test service and execute these tests. However, CI cannot currently execute because a Gitea Actions runner has not yet been provisioned for the repository by the course infrastructure.

The shared Supabase development database was deliberately not used as a disposable integration-test target.

That is much better than falsely putting:

CI 

when there was never a runner.

And yes, we're still heading directly toward:

#58 

↓

#62 ← finish this PR

↓

#47 

Once #62 is reviewed and merged, **#47 is the next issue I want you to work on**, assuming #46/schema dependencies haven't changed in the meantime.

8/9/2026, 3:59:45 PM



what abt index iun first commit?

8/9/2026, 4:00:47 PM



Yes — include **both relevant** `index.ts` **files in the first commit**, but for different reasons.

`apps/backend/src/database/index.ts` is already included automatically when you run:

```
git add apps/backend/src/database
```

because that stages the entire new `database/` folder, including:

```
apps/backend/src/database/  
├─ errors.ts  
├─ index.ts      ← included  
├─ pool.ts  
├─ query.ts  
└─ transaction.ts
```

Also include:

```
apps/backend/src/index.ts
```

because we changed it to perform graceful shutdown and close the database pool. That belongs with the database connection foundation rather than the repository/docs commit.

So your **first commit staging commands** should be:

```
git add apps/backend/src/database  
git add apps/backend/src/index.ts  
git add apps/backend/tests/database/transaction.database.test.ts  
git add apps/backend/tests/unit/database-errors.test.ts
```

Then check:

```
git diff --cached --stat
```

You should see both:

```
apps/backend/src/database/index.ts
apps/backend/src/index.ts
```

They are two separate files:

```
src/database/index.ts
    ↑
exports the reusable DB utilities

src/index.ts
    ↑
starts/stops the backend application
    and closes the DB pool on shutdown
```

So yes: **both belong in Commit 1.**

8/9/2026, 4:00:55 PM